

Microprocessor System Project 2DX3

Final Project Report

COMPENG 2DX3

McMaster University
Hamilton, ON

Vaishnav Jayaraj (jayarajv)
400588635

Statement of Originality: I hereby certify that all the work described within this report is the original work of the author. Any published (or unpublished) ideas and/or techniques from the work of others are fully acknowledged in accordance with the standard referencing practices, and I have obtained the necessary permission from the company to submit this report.

Table of Contents

List of Figures.....	2
List of Tables	3
1. Device Overview	4
1.1 Features.....	4
1.2 General Description	5
2. Device Characteristics Table	7
3. Detailed Description.....	8
3.1 Distance Measurement	8
3.2 Visualization	9
4. Detailed Description.....	10
4.1 Note.....	10
4.2 Instructions	10
4.3 Expected Output.....	13
5. Limitations.....	16
5.1 Floating Point numbers and trigonometric capabilities.....	16
5.2 Stepper motor speed limitations.....	16
5.3 Serial communication rate	16
5.4 Communication methods and speed limitations	17
6. Circuit Schematic:.....	18
7. Programing Logic Flowchart:	19

List of Figures

Figure 1: Image of final assembled design	6
Figure 2: Block Diagram (Data flow graph).....	6
Figure 3: Code controlling net resolution of the scan	10
Figure 4: Sequence to translate, build, and load code onto MCU.....	10
Figure 5: Method to identify specific port used	11
Figure 6: MATLAB variables controlling port communication and resolution.....	11
Figure 7: Image of hallway in McMaster JHE building to be scanned.....	13
Figure 8: Scatter Plot of Scan	14
Figure 9: Wireframe mesh of scan	14
Figure 10: ZY axis of Wireframe mesh scan.....	15
Figure 11: XY axis of Wireframe mesh scan	15
Figure 12: Maximum serial communication rate.....	16
Figure 13: Circuit Schematic of Design.....	18
Figure 14: Flowchart of entire system design.....	19

List of Tables

Table 1: Connection for each component and microcontroller settings.....	7
--	---

1. Device Overview

1.1 Features

The microcontroller Texas Instrument MSP432E401Y and the time of flight 9ToF) sensor were the two most essential components for the implementation of the project. The features of the devices are as follows:

Texas Instrument MSP432E401Y microcontroller:

- Processor: Cortex-M4F Processor with Floating – Point Unit(FPR), Harvard Architecture
- Bus Speed: 26MHz (Assigned speed)
- Memory: 1024 KB flash memory
- Operating Voltage: 2.5V to 5.5V
- SRAM: 256KB
- EEPROM: 6KB
- Analog to Digital: SAR based two 12-bit ADC modules
- Programming Language: Assembly or C language using Keil IDE
- Communication: UART and I2C – Serial Communication
- Baud Rate: 115200bps (for UART between microcontroller and PC)
- I/O: 2 onboard push buttons and 3 on-board LED's
- Cost: \$75.00 CAD (Mouser electronics)

Time of Flight Sensor (ToF) – VL53LIX:

- Maximum Measurable distance: 360cm
- 3 discrete distance modes: short, medium, and long range
- Operating voltage: 2.6V to 3.5V
- Communication: I2C with a maximum frequency of 400KHz – Serial Communication
- Cost: \$27.00 CAD (Mouser electronics)

Stepper Motor (28BYJ-48) & ULN2003 Driver:

- Performs a full rotation of 512 steps (1 step = 4 individual full drive phase transitions)
- 4 LEDs to indicate current phase
- Operates at a voltage between 5 – 12 V
- \$8 CAD

Scanning/Visualization:

- Code written in Keil to perform plane scans
- Visualization of data points using MATLAB

Miscellaneous components:

- Push Buttons – Control state of motor and to start scanning
- Resistor – 1 K Ω pull up resistors used to implement the push button
- Onboard microcontroller LED – Used to signify scan start, measurement status, and data transmission state.

1.2 General Description

The purpose of the project was to map out a 3D space (hallways, enclosed surfaces, small rooms) and generate a 3D model of it. To achieve this, the MSP432E401Y was used as the microcontroller to control, read, and process data from the VL53L1X ToF sensor and Stepper motor which performed the measurements. The raw data collected by the ToF sensor was then processed and visualized using MATLAB to create a 3D map.

The VL53L1X ToF uses LIDAR techniques to acquire the measurements. The onboard emitter sends a light beam which is reflected by an object/surface and the distance is read. The ToF sensor is mounted on the stepper motor using a custom 3D printed mount which was designed using AutoCAD. The stepper motor continuously rotates with the ToF sensor motor mounted and stops periodically to take measurements. The number of measurements taken and hence number of angle rotations can be customized by the program, however by default there are 32 measurements per rotation resulting in a step angle of 11.25 degrees.

Each analog measurement is converted into a digital measurement by the ToF and is transmitted to the microcontroller using I2C communication protocol. This data is then packed and serially transmitted to the computers serial port (COM4) using the UART protocol. The baud rate during this communication is 115200 bps. Since the system can only map 2D planes with each rotation, the entire device must be manually shifted in the direction perpendicular to the plane. The specific distance of this shift is dependent on the user's required resolution accuracy (intended it be incremented 10cm at a time).

The received data is then plotted on a 3D graph to give an approximate visual of the special composition. The z and y coordinates are calculated numerically using trigonometric properties involving the distance value of the motor angle, with the x coordinate incrementing by a fixed value. This info is then used to plot a scatter plot on MATLAB. A wire mesh generator software was also developed in MATLAB to connect each layer give a clearer visualization of the mapped space.

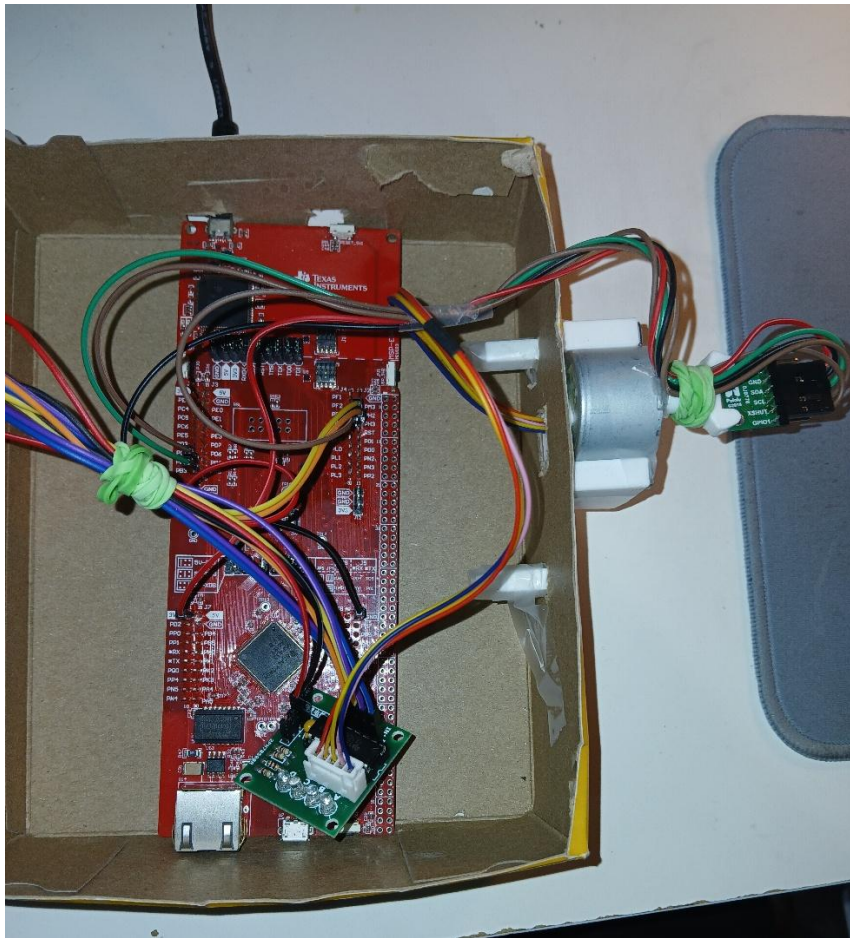


Figure 1: Image of final assembled design

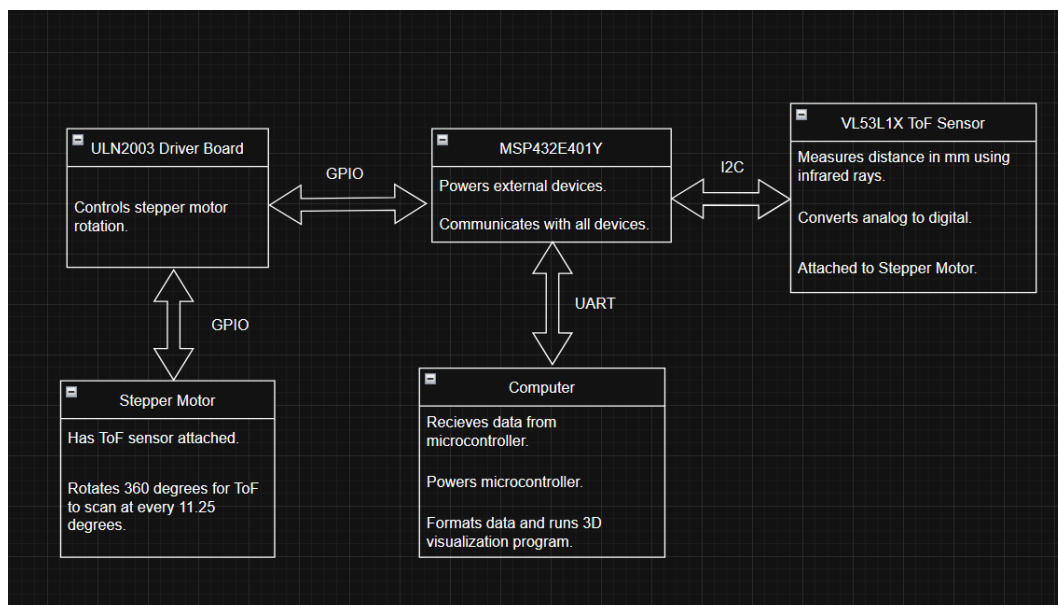


Figure 2: Block Diagram (Data flow graph)

2. Device Characteristics Table

Feature	Value
Bus Speed	26MHz
Digital I/O LED Status	LED 1 (PN1): Toggles when start scan button (PJ0) is ready to press LED 2 (PN2): Toggles when transmitting data through UART. LED 4 (PF0): Toggles when measurement is taken.
Microcontroller Pins	Stepper Motor: PH0 to PH3 I2C: PB2 and PB3 Buttons: PJ0 Power: 5V, 3.3V and GND
Stepper Motor Pins to Microcontroller	IN1:IN4 → PH0:PH3 V+ → 5V V- → GND
ToF Sensor Pins to Microcontroller	Vin → 3.3V GND → GND SDA → PB2 SCL → PB3 XSHUT → PG0
Baud Rate	1152200 bps
Serial Port	COM4

Table 1: Connection for each component and microcontroller settings

3. Detailed Description

3.1 Distance Measurement

The VL53L1X Time-of-Flight (ToF) sensor plays a critical role in measuring distances. It works by emitting infrared light pulses from its emitter. The light then travel to an object obstructing the sensors path, where it would then reflect back into the sensor's receiver. The time delay between emitting the light and receiving it is measured and used to calculate the objects distance. This is done using the time-of-flight formula:

$$Distance = \frac{Time\ of\ Flight \times Speed\ of\ light}{2}$$

Since the speed of light is a known and fixed value, the distance can easily be calculated by multiplying the measured value by half the speed of light.

After receiving the distance data, the ToF sensor will internally start processing the signal with the necessary steps: transduction, conditioning, Analog to Digital conversion, and outputting a digital value representing distance in millimetres. This data is then sent to the microcontroller via I2C communication.

Upon startup of the microcontroller, it will invoke all the setup and initialization functions. These include the initialization of the ToF sensor, clock settings, onboard LED's, I2C and UART communications, as well as GPIO ports and interrupt initialization. As mentioned, the interrupts were used for the start button initialization. The onboard button (PJ0) was initialized as an interrupt. This means that whenever PJ0 is pressed, the system will finish its process and immediately perform the instructions assigned to the button press. A combination of interrupts and polling were used for the design. When the button (PJ0) is pressed, the microcontroller would enter the interrupt handler, and set a start scan flag to true. The microcontroller would already be polling, waiting till this start flag is set true. When it recognises it as true the scanning process begins.

As the ToF sensor starts scanning, the stepper motor will start rotating, allowing for the sensor to scan at different positions. The stepper motor has a total of 512 steps to cover 360 degrees. The measurements are taken every 11.25 degrees, which corresponds to the default 32 segments/measurements taken per rotation. To ensure that the scan captures aa clear reading, the system pauses every time the motor reaches a new measurement angle. For every point that the sensor captures, the data is logged and sent to the computer as a distance reading, phase angle, and set x axis. Furthermore, status LED's flash whenever a measurement is taken (PF0), when data is transmitted through UART (PN2) and finally when the start button is ready to press (PN1).

To prevent wire twisting and avoid excess cable tension and tangling, the motor makes a complete anticlockwise revolution after each layer scan. This also allows for a brief period to move the circuit forward and to make any measurement adjustments. As mentioned previously the x/tangent displacement to each layer is a set incremental amount which must be manually measured and moved. The remaining z and y coordinates can be computed by using trigonometry to convert the radial distance and motor angle form polar to cartesian form.

3.2 Visualization

Whenever the microcontroller is reset, a MATLAB code for collecting and visualizing the data is manually run. When the start button is pressed, it sends a start message through UART to the computer. This then signals the MATLAB code to start recording data.

For every data point collected, the microcontroller sends a data package outputting the distance measured (r), phase angle (θ), and current layer (x_{layer}). The MATLAB code reads each data transmission line by line and parses the data to extract the floating point numbers from the text. The polar coordinates are then converted to cartesian using the following formulas:

$$x = x_{layer} \quad y = r \cos(\theta) \quad z = r \sin(\theta)$$

With this, each converted coordinate is stored in pre-allocated arrays. Each (x,y,z) coordinate is then plotted real-time on a 3D cartesian plot. After all the scans are finished an end signal is sent to the computer which will indicate the code to stop reading the communication port. After the scatter plot is made, another plot is made using the data arrays which uses approximate geometry calculations to connect each data point in a layer and stack the layers to simulate a wireframe or surface mesh of the space.

4. Detailed Description

4.1 Note

This special measurement system offers a solution for various applications where an accurate 3D mapping of an enclosed surface is required. Potential applications include mapping buildings for navigation and security, analysing mappings of smaller or harder to reach structures and surfaces, as well as visualization for augmented/virtual reality. Overall, this system an efficient and cost-effective solution, allowing for users to acquire, analyse, and visualize special data with precision and efficiency. Its use of different hardware components, programming, and modules enables seamless operation and provides the opportunity to be included into larger scale systems to suit diverse application needs.

4.2 Instructions

- 1) Ensure all connections are made according to the circuit schematic in Fig 13.
- 2) Ensure that the required software (Keil μ Vision, MATLAB) are downloaded and are up to date.
- 3) Open the “2dx_studio_8 μ Vision” project file. Once the IDE is opened, double click and open the “2dx_studio_8.c” file. Ensuring that the microcontroller is connected to the PC and the correct debugger settings are chosen. Adjust the MEASUREMENTS_PER_TURN and NUM_OF_SCAN_LAYERS values in line 24 and 26 to however many probing measurements per layer and number of layers needed. If not keep default values. Refer to the figure below:

```
22 #define MAXRETRIES 5 // number of receive attempts before giving up
23 #define FULL_ROTATION_STEPS 512 //Every motor rotation func takes 4 steps (2048=Total # of steps) (2048/4=512)
24 #define MEASUREMENTS_PER_TURN 32
25 #define STEPS_PER_MEASUREMENT (FULL_ROTATION_STEPS / MEASUREMENTS_PER_TURN)
26 #define NUM_OF_SCAN_LAYERS 3
27
```

Figure 3: Code controlling net resolution of the scan

- 4) Click Translate → Build → Load. Refer to the figure below for the buttons:

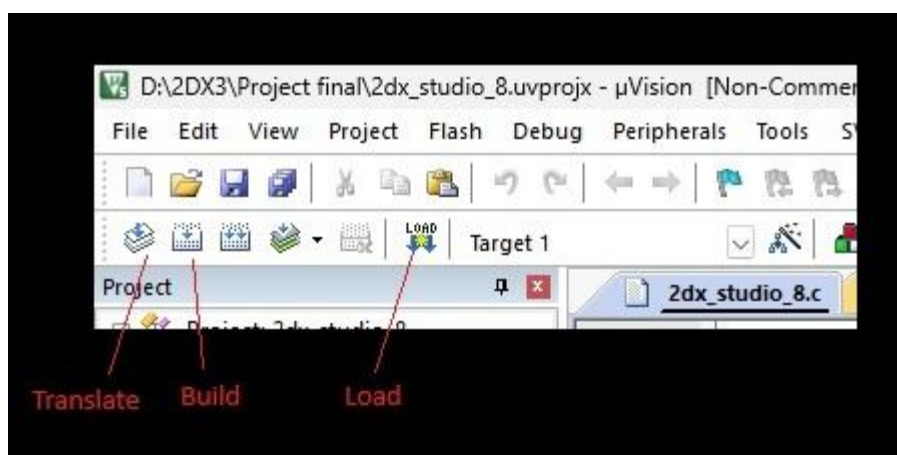


Figure 4: Sequence to translate, build, and load code onto MCU

- 5) Next go to the search bar of your computer and search for Device Manager and click enter. Look under Ports (COM & LPT) and note down the COM number next to XDS110 Class Application/User UART. The example below shows COM 4, but each computer will have a different number. Refer to the figure below:

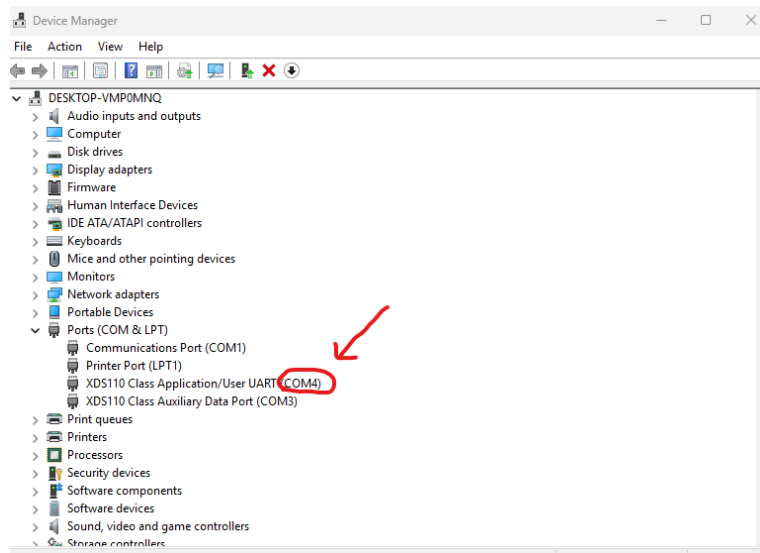


Figure 5: Method to identify specific port used

- 6) Open the MATLAB file “DX3_Final_Project_code.m”, go to line 8 and change the port to your specified COM port. Furthermore, if the number of layers or points per rotation were altered before, lines 20 and 21 must be changes to match the values in the Keil project file.

```

DX3_Final_Project_code.m x
D:\2DX3\Project final\DX3_Final_Project_code.m
1  %% =====
2  %% Real-Time 3D Scan
3  %% =====
4
5  clear;
6
7  %% SERIAL CONFIGURATION
8  port = "COM4"; % change to your COM port
9  baudrate = 115200;
10
11  device = serialport(port, baudrate);
12  device.Timeout = 50;
13  configureTerminator(device, "CR/LF");
14  flush(device);
15
16  fprintf("Opened %s\n", port);
17
18  %% PARAMETERS
19  %Change if doing less scans
20  num_layers = 3; % total scans/layers expected
21  points_per_rotation = 32; % measurements per full rotation
22  max_points = num_layers * points_per_rotation;
23

```

Figure 6: MATLAB variables controlling port communication and resolution

- 7) Run the MATLAB code and press the reset button on your microcontroller.

- 8) Carefully observe LED1. When it flashes twice after the initial startup, it indicates that the start button is ready to be pressed. Press the onboard button PJ0 and allow for the first plane scan to take place.
- 9) Observe the plot and ensure that there are no moving obstructions in the environment.
- 10) After one revolution, the motor will unwind. During this time move the device by a constant displacement in the x axis.
- 11) Allow for the next scan to take place and repeat steps 9 to 10 until all the scans are done
- 12) Observe and analyse the final plot

4.3 Expected Output

Below is an image of the hallway that was scanned for the purpose of demonstration. Observe that the hallway widens on the left by a foot. There is also an industrial elevator shaft of the right with a junction opening left and right after it.



Figure 7: Image of hallway in McMaster JHE building to be scanned

The following images show the special scan of the hallway with 128 scans per layer with a total of 24 layers scanned, each displaced by a foot.

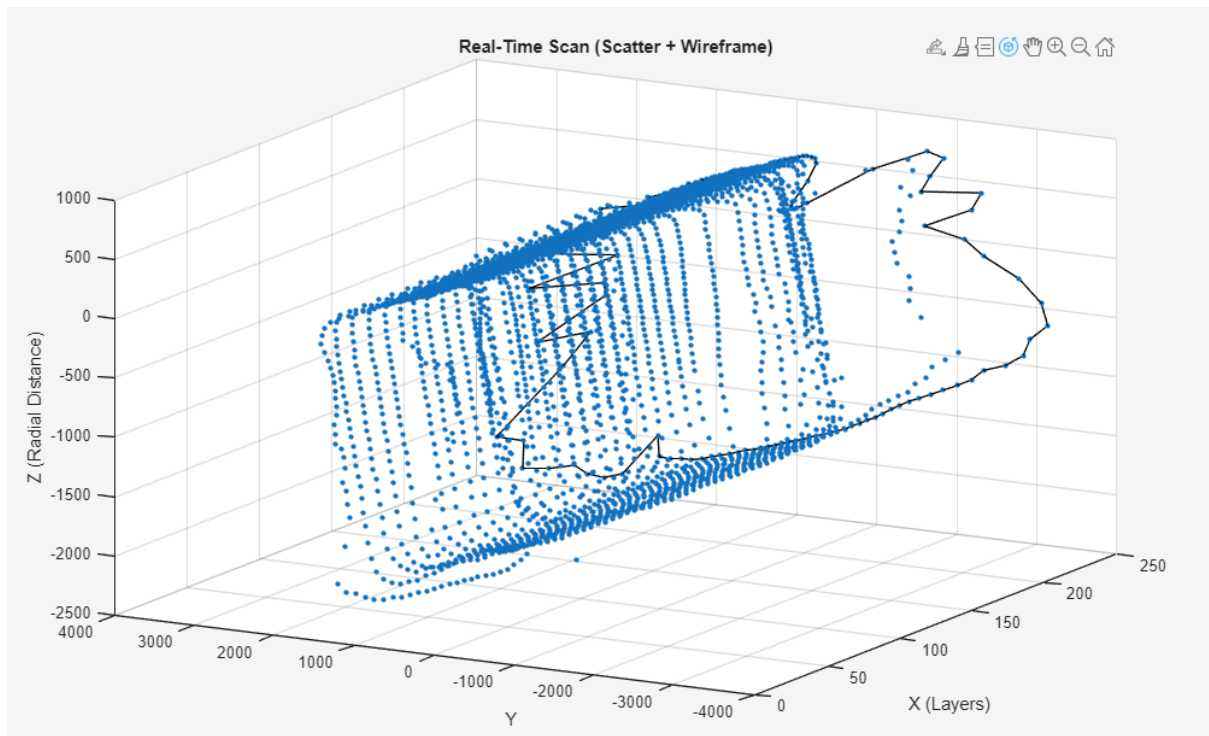


Figure 8: Scatter Plot of Scan

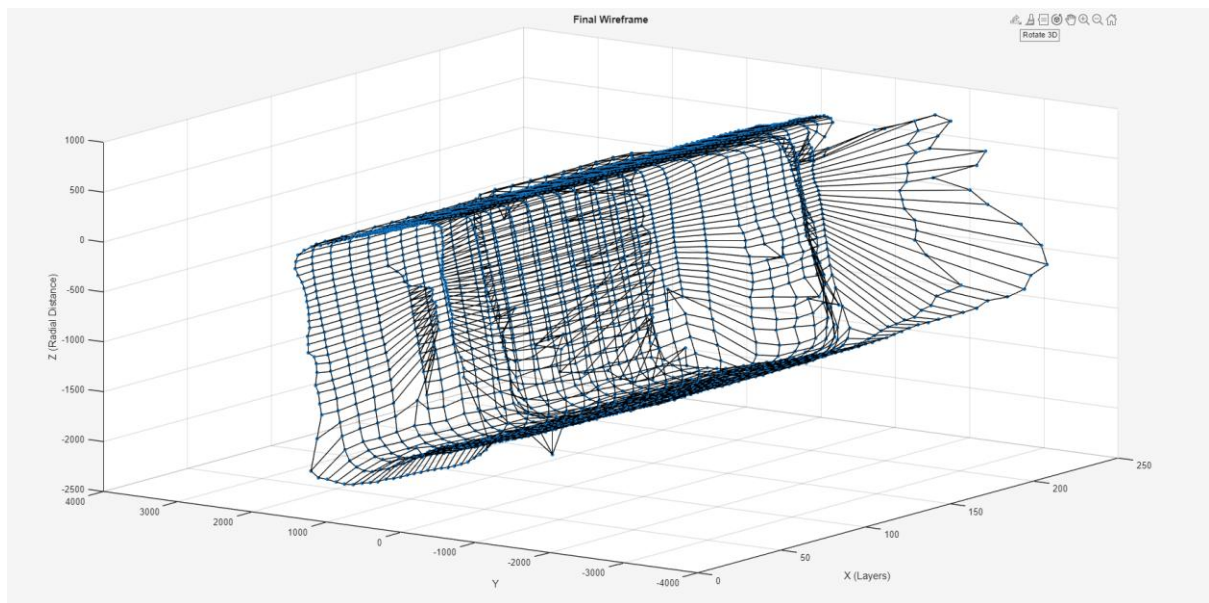


Figure 9: Wireframe mesh of scan

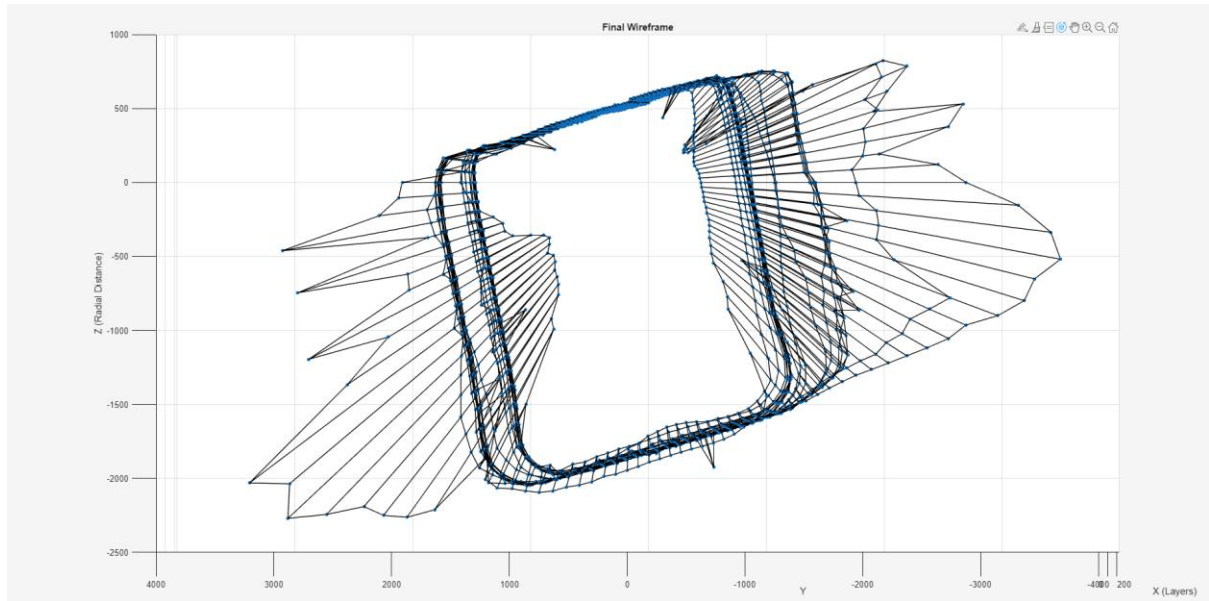


Figure 10: ZY axis of Wireframe mesh scan

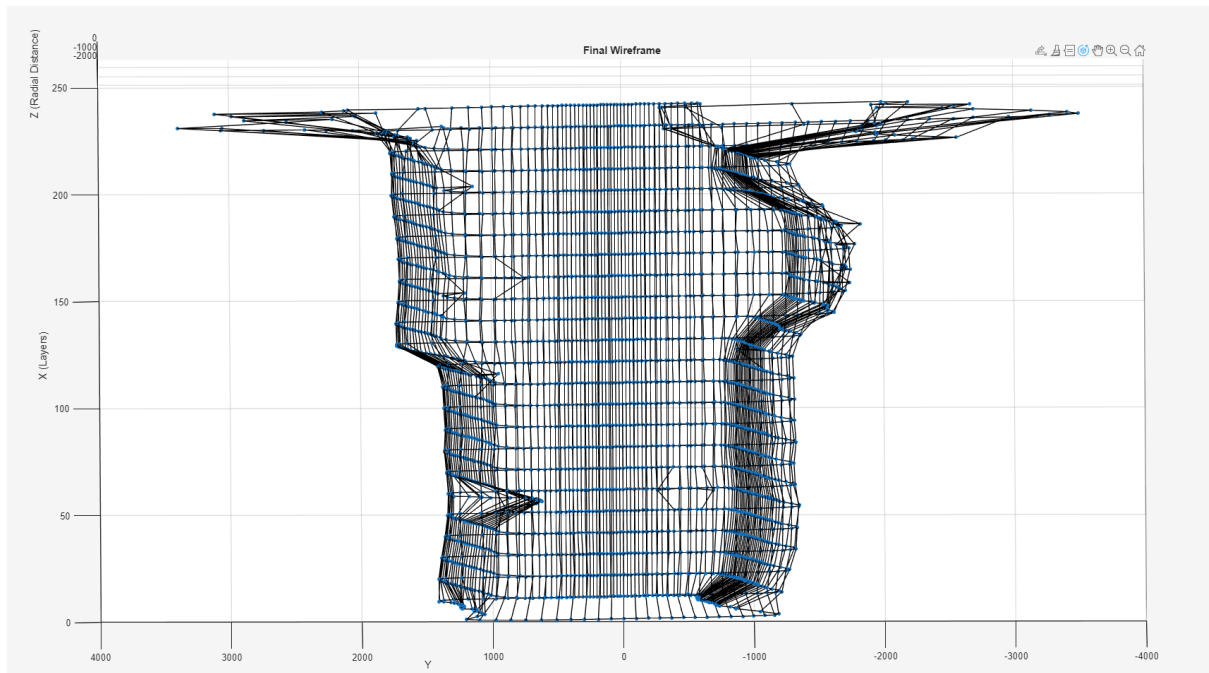


Figure 11: XY axis of Wireframe mesh scan

From the figures, all the characteristics of the hallway can be clearly seen including the indent of the left, the elevator, and the junction opening at the top. The general shape and properties are all captured and shown in extremely good detail.

5. Limitations

5.1 Floating Point numbers and trigonometric capabilities

Although the microcontroller has a prominent Floating-Point Unit (FPU) capable of handling single-precision 32-bit operations such as multiplication, division, subtraction and addition; to ensure higher precision, sin and cos functions were computed using MATLAB, which would provide double the precision (64-bits). While this makes the development process faster, it disregards potential optimizations offered by onboard processing.

5.2 Stepper motor speed limitations

The speed of the motor's rotation is limited. If the motor is spun too fast it can result in the motor stalling due to the rapid changing speed as well as overheating and permanent damage of the motor. The stopping of the motor between scanning and rotating stage also majorly contributes to overall speed of the system.

5.3 Serial communication rate

Serial communication between the microcontroller and computer is constrained by the PC's maximum standard serial communication rate, set at 128,000 buts per second. This can be checked for any computer from the device manager → Ports → Right click on port → Properties → Port settings:

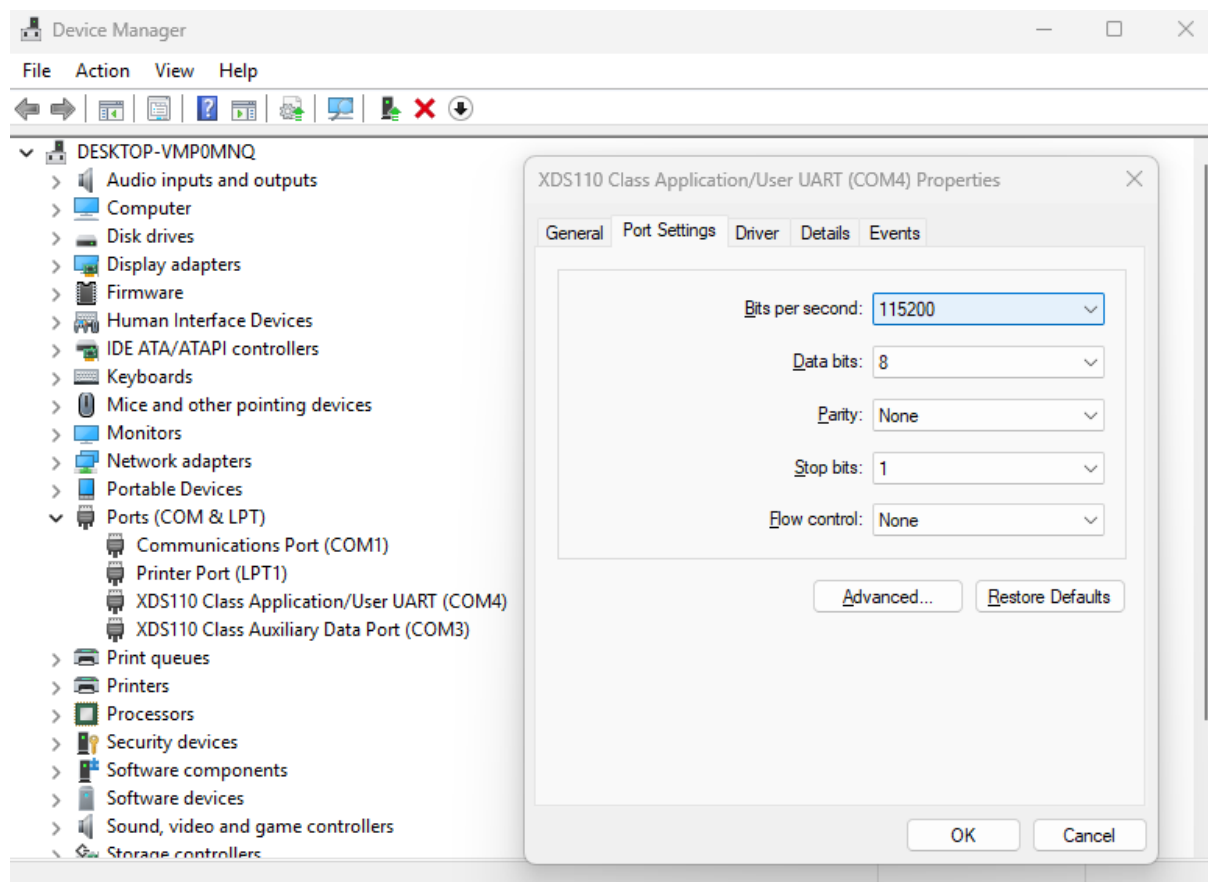


Figure 12: Maximum serial communication rate

Despite this the microcontroller only supports speeds up to 15Mbps, the final rate is based on the PC's limitations. This constraint highlights the importance of compatibility considerations in system design. The baud rate that was used was 115200 bps.

5.4 Communication methods and speed limitations

The device uses I2C protocol for communication between the microcontroller and ToF sensor, which gives a maximum transmission speed of 3.33Mbps. However, the UART sensor runs on a baud rate of 114200 and bus speed of 26MHz. Furthermore, the ToF has a maximum ranging frequency of 50 Hz and a UART and I2C have set bus speed and baud rates. These speed rates limit the maximum speed that can be achieved in transmission to make the scans faster.

6. Circuit Schematic:

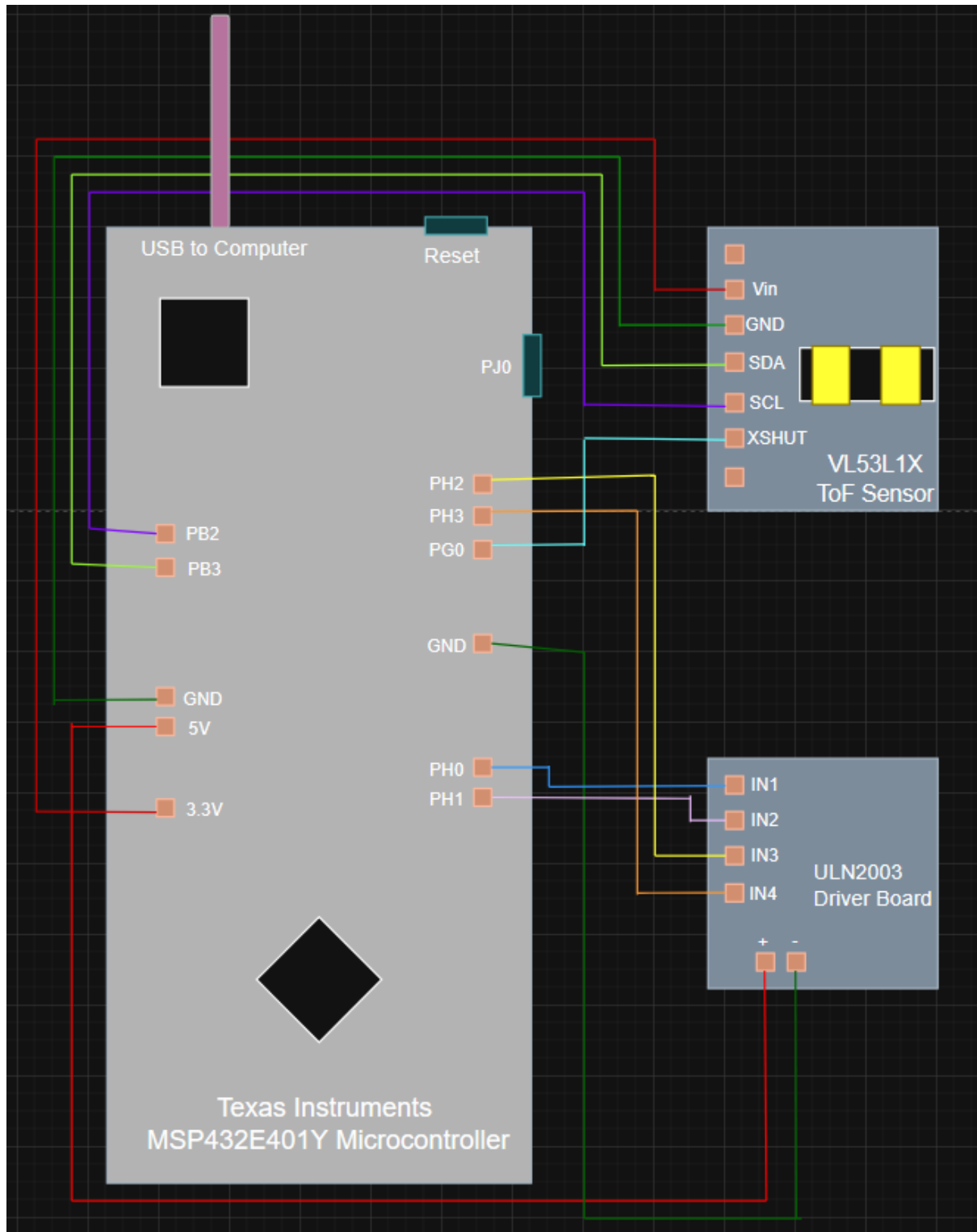


Figure 13: Circuit Schematic of Design

7. Programing Logic Flowchart:

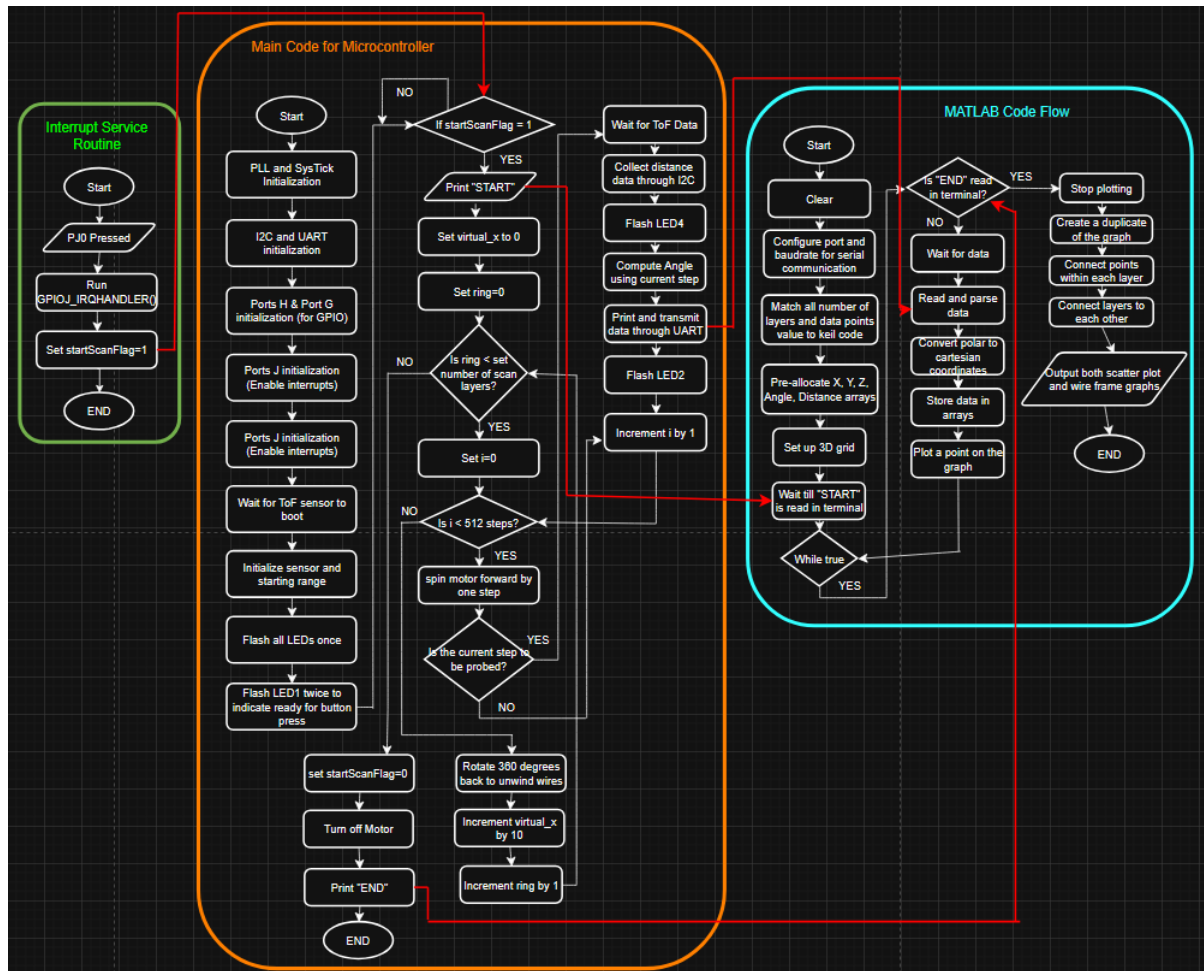


Figure 14: Flowchart of entire system design